# phonefleet + Chipiron

Controlling the motor from laptop + phone (scripts v3)

Matilde

October 23, 2026

Current pipeline to run an ESCON motor through Arduino/Python and laptop/phone, with the **v3** scripts. The full file list and dependencies are in `README_chipiron_v3.md`). **CAUTION:** nothing has been tested on Chipiron itself so far, and **The LoRa link has not been tested at all** (Section 5).

All up-to-date scripts are here: https://github.com/Turbotice/icewave/tree/main/icewave/chipiron

**Setup used**

Windows laptop, VS Code/git, Fairphone (FP4) running the Gobannos app, laptop connected to the phone's Wi-Fi hotspot.

## Contents


**1 Overview and pipeline** **3**

1.1 ESCON Studio: load the parameters . . . 5

1.2 Arduino IDE: flash `PWM_full_v3.ino` . . . 6

1.3 Phone: plug the Arduino, Gobannos, hotspot . . . 7

1.4 Laptop: connection and IP settings . . . 7

1.5 Which test? . . . 8

1.6 Run the session: `chipiron_session_v3.py` . . . 9

1.6.1 Launch and sequence . . . 9

1.6.2 Choice 1: manual . . . 10

1.6.3 Choice 2: oscillation (firmware or stream) . . . 10

1.6.4 Choice 3: run a program (constant speed, ramps, steps, cycles, CSV) . . . 11

1.6.5 Choice 4: another script . . . 13

1.6.6 Running a script without the session . . . 13

1.7 Stop, list and save the files . . . 13

1.8 Decode and plot: `chipiron_plotting_v3.py` . . . 14

1.8.1 How the Arduino data reach the files and are decoded . . . 14

1.8.2 Use . . . 15
1.8.3 Coding and decoding explained . . . 17
1.9 End of the test and diagnosis . . . 18
1.10 Scripts at a glance . . . 18
1.11 Optional (not fully tested): SSH and Termux . . . 19

**2 Ways to send an input** **22**
2.1 Command grammar (what the Arduino understands) . . . 22
2.2 Channels . . . 22

**3 Notes for a first test with Chipiron** **23**
3.1 Safety and pre-checks . . . 23
3.2 Identify A1 and A2 (who is speed, who is current) . . . 23
3.3 Basic tests by URL (no script) . . . 24
3.4 Session test 1: the basic chain (manual) . . . 24
3.5 Session test 2: constant speed, ramps and steps . . . 24
3.6 Session test 3: a ramp-planner CSV . . . 25
3.7 Session test 4: oscillation, firmware mode . . . 25
3.8 Fault and safety tests . . . 25

**4 Sine skewing debugging (lab motor, 21–22 October 2026)** **26**
4.1 The model: a first-order lag . . . 26
4.2 The two fit cells in `chipiron_plotting_v3.py` . . . 27
4.3 Log of the tests and what each one gave . . . 28
4.4 Speed quantisation and the ESCON "averaged" outputs . . . 29
4.5 Procedure for the real motor and load . . . 30
4.6 Open points . . . 31

**5 Current limitations of the scripts** **32**

**6 Field warnings and notes (with the real Chipiron)** **33**
6.1 Messy notes . . . 33
6.2 Oscillation tests: physical target . . . 33

# 1 Overview and pipeline

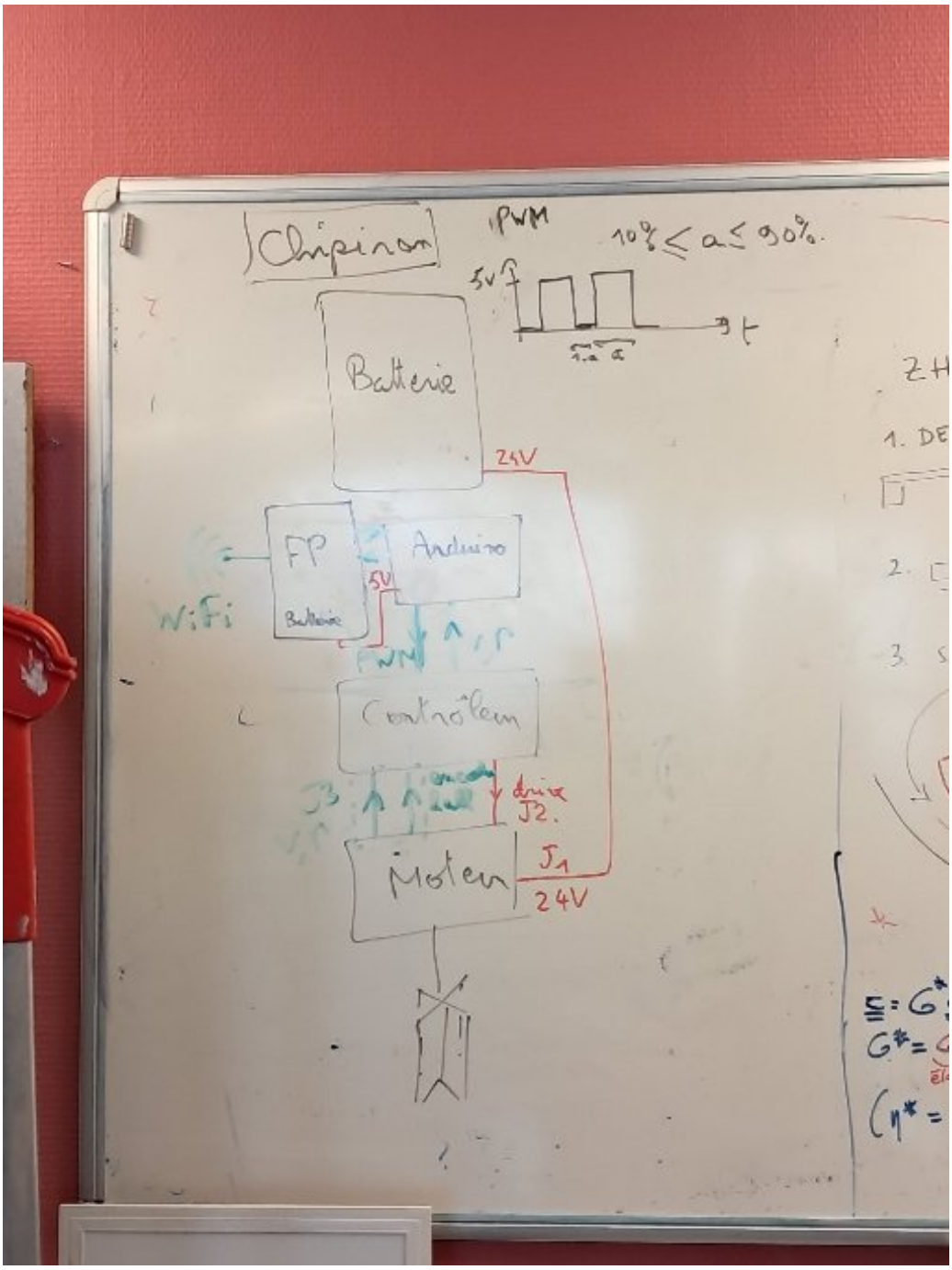



**Figure 1:** Chipiron overview (sketch by Stéphane)

**Overall procedure:**

- The laptop sends HTTP requests (`/start`, `http:<phoneIP>:8080/usb-cmd/<command>`, `http:<phoneIP>:8080/usb-cmd/e` (enable motor), `http:<phoneIP>:8080/usb-cmd/d` (disable motor), `/stop`, `/status`, `/list-files`, `/get-file/<name>`).
- The **Gobannos app on the phone** is the HTTP server (port 8080). It forwards `<command>` (`mo`, `c60.0`, `o50,20,0.2`, `x`, `e`, `d...`) to the Arduino over USB, records the sensors of the phone and logs the bytes sent back by the Arduino (the "Usb" file).
- The `PWM_full_v3.ino` firmware is flashed on the **Arduino Mega** from the Arduino IDE on a

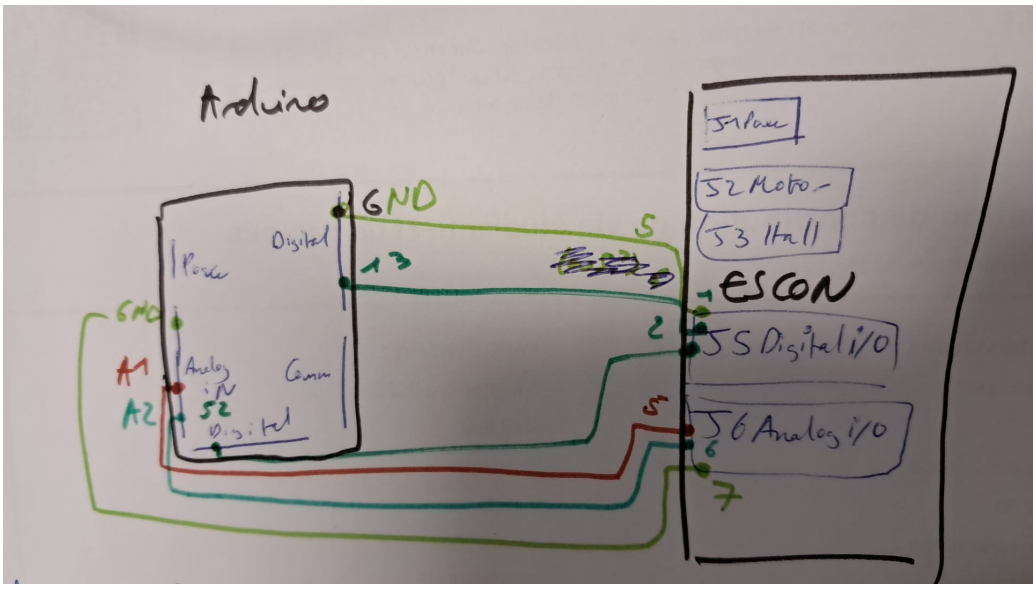



**Figure 2:** Arduino/ESCON controller plugging (lab EC motor).

laptop (board *Arduino Mega 2560*), only once before running experiments. It outputs a PWM duty cycle on pin 13 (ESCON set value), drives the enable line (pin 52) and sends back two analog readings (A1, A2) every 250 ms.

- The **ESCON 70/10** turns the duty cycle into a speed set value using the mapping stored in its `.edc` parameters. The *same* `.edc` file is read by the Python scripts, so that rpm ↔ duty conversions, the rest duty (duty that gives 0 rpm) and the ESCON speed ramp limits are always those of the controller.

Below: step-by-step route for one test. Termux is **not** used for the experiment (Gobannos only); it is kept for diagnosis (Section 1.11). Optional steps (parameter patching, ramp planning, manual checks, clock synchronisation, offline decoding) are shown dashed in Figure 3 and described in the subsection with the same number or in the one indicated.

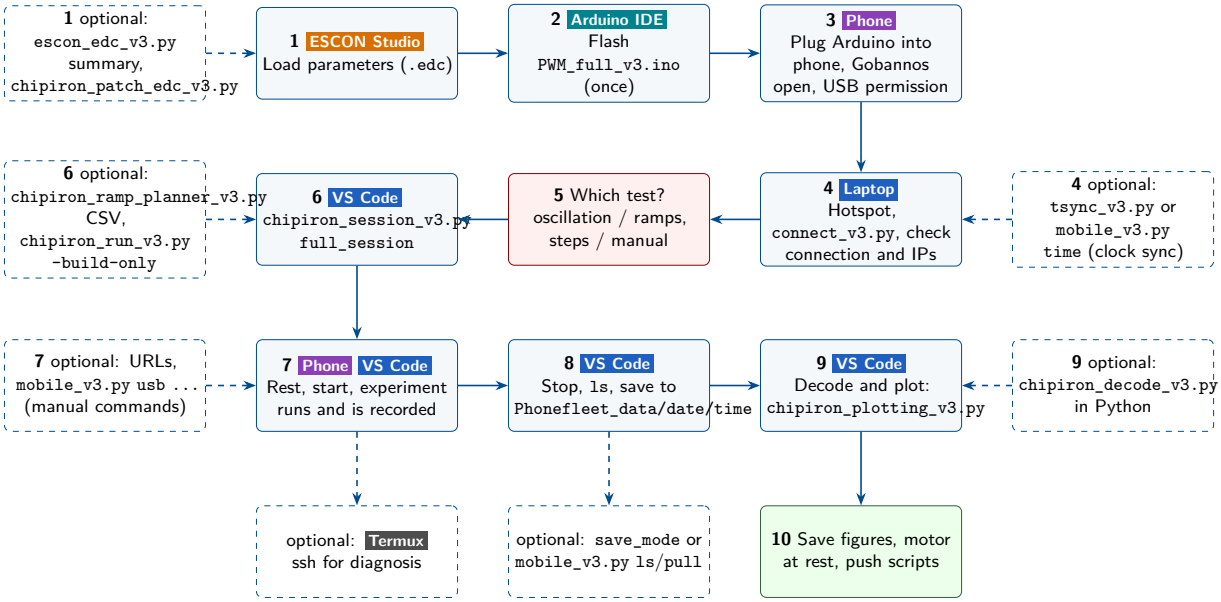



**Figure 3:** Pipeline of one Chipiron test. Dashed boxes are optional or for quick tests and debugging. The numbers match the subsections of this section.

## 1.1 ESCON Studio: load the parameters

ESCON Studio VS Code (optional tools: `escon_edc_v3.py`, `chipiron_patch_edc_v3.py`)

- **Read** an `.edc` before loading it: `escon_edc_v3.py` prints the motor type, operation mode, PWM mapping (value at 10, 50 and 90 %, duty giving 0 rpm, polarity), speed ramp, and the analog-output conversion (volts $\rightarrow$ rpm and A). It also warns about inconsistencies (for instance an offset adjust stored but not enabled). The same file is read by every script (session, run, oscillate, planner, plotting).
- **Edit** (a copy of) an `.edc` with `chipiron_patch_edc_v3.py`: analog-output shift to 0–4 V, bipolar set value, speed ramp, motor type, offset. Always **open the result in ESCON Studio and check every value** before writing it to the controller.
- Keep the controller **disabled** while loading the `.edc`; double-check every parameter in the software and edit by hand if required.
- The ESCON configuration must match the Arduino mode of the test (Section 1.5): **bipolar** (`mo`: 10 % $\rightarrow -800$ rpm, 50 % $= 0$ rpm, 90 % $\rightarrow +800$ rpm) or **unipolar** (`mc`: 10 % $\rightarrow 0$ rpm, 90 % $\rightarrow 800$ rpm). With a bipolar `.edc`, ramps and steps run in `mo` too (`c<duty>` is accepted in both modes).
- Optional: keep the real-time speed/current display visible in ESCON Studio (*Tools* $>$ *Data recorder* $>$ *right click on graph to export data*; only the current time window is exported).

Terminal (VS Code): read an .edc

```
py escon_edc_v3.py parameters_Sep26.edc --folder edc_summaries
py escon_edc_v3.py parameters_Sep26.edc --no-save
py escon_edc_v3.py
```

The first line prints the summary and saves it as `.txt` in `edc_summaries`; the second prints only; the third asks for the file and whether to save.

Terminal (VS Code): build a patched copy

```
py chipiron_patch_edc_v3.py --in parameters_Sep26.edc --out parameters_Sep26_v2.edc --
    ↪ bipolar-setpoint 800 --ramp 2000 --motor-type DC --offset -11
```

| Argument | Meaning |
|---|---|
| `-in FILE`, `-out FILE` | source `.edc` and the *new* file (must not exist, unless `-force`; must differ from `-in`) |
| `-ao-shift` / `-no-ao-shift` | remap the two analog outputs to 0–4 V so that the Arduino (0–5 V ADC) can read negative values: speed $-1000 \ldots +1000$ rpm $\rightarrow$ 0–4 V, current $-5 \ldots +5$ A $\rightarrow$ 0–4 V. **On by default and required with the bipolar mapping** (see the box below). Use `-no-ao-shift` only if the file already has the shift (e.g. a file whose name ends in `_ao_shift`) |
| `-speed-range R`, `-current-range I` | rpm and A at full scale of the AO shift (defaults 1000 and 5) |
| `-bipolar-setpoint R` | PWM set value becomes bipolar: 10 % $= -R$, 50 % $= 0$, 90 % $= +R$ rpm |
| `-ramp RPM_PER_S` | ESCON speed ramp (acceleration and deceleration) |

| Argument | Meaning |
|---|---|
| `-motor-type EC\|DC` | written *only* when given. Enum of the `.edc`: 0 = EC (brushless), 1 = DC (brushed); a wrong value can make the controller drive the motor wrongly |
| `-offset RPM` | fixed set-value offset (can be negative). It only acts if the offset functionality is *enabled* in ESCON Studio; the script prints the state of the enable bit |
| `-speed-p-gain N` | speed controller P gain (integer, no unit, as shown in ESCON Studio). Written to the regulator actually in use: Speed 1 (mode of controller "Current Limiter") or Speed 2 ("Inner Current Control Loop"); the script says which one. Too low a gain flattens /skews fast sines; too high gives noise or oscillation: raise it step by step |
| `-force` | overwrite `-out` |

.

**Bipolar mapping ⇒ use the AO shift**

With bipolar PWM mapping the motor reverses, so the actual speed and current become negative. The analog outputs of `parameters_Sep26.edc` as delivered are *not* shifted ($\pm 4$ V around 0 V): a negative speed would be a negative voltage on the Arduino pin, which reads it as 0 (and a pin pushed below about $-0.5$ V is outside the ATmega ratings). The AO shift (default of the patch script) maps the outputs to 0–4 V with 0 rpm and 0 A at 2 V:

$$\text{speed} = \frac{V-2}{0.002}\ \text{rpm}\ (\pm 1000\ \text{rpm range}), \qquad \text{current} = \frac{V-2}{0.4}\ \text{A}\ (\pm 5\ \text{A range}), \qquad V = \text{adc}\cdot V_{\text{ref}}/1023.$$

Choose `-speed-range` at least as large as the maximum speed: 4 V corresponds to +range (the ADC still reads up to 5 V, i.e. 1.25 times the range, then saturates) and one ADC count is $\approx 0.0024 \times$ range rpm (2.4 rpm for 1000 rpm, about 6 rpm for the 2460 rpm of the Chipiron configuration). **Which AO feeds which Arduino pin is hardware** (AO1 → A1, AO2 → A2, current on AO1 and speed on AO2 in the lab, Section 3.2); **which quantity each AO outputs is set in the `.edc`** (`PARAM_AnalogOutputsConfiguration`, here AO1 = Actual Current, AO2 = Actual Speed).

**Procedure: offset**

The offset should be tuned so that the rest command (`c50.0` in bipolar mapping) gives nearly 0 rpm. Tune it by hand in ESCON Studio (send `c50.0` and check the speed), then use `chipiron_patch_edc_v3.py -offset` to store it, **and check that the offset adjust is enabled** in ESCON Studio.

**Debugging and warnings**

If in ESCON Studio the "PWM value" is stuck at an out-of-range value and the motor is stuck in disabled/error mode, check that the Arduino is actually plugged in and running (a reset Arduino now boots with the motor disabled until it receives `mo`, `mc` or `e`).

## 1.2 Arduino IDE: flash `PWM_full_v3.ino`

Arduino IDE Board *Arduino Mega 2560*, flashed from the laptop **once**; afterwards unplug it from the

laptop and plug it into the phone. Two running modes are chosen at run time by a text command.

| Command | Mode | Effect |
|---|---|---|
| `mc` | constant | select constant mode (rest duty 10 %), go to rest, **enable the motor** |
| `mo` | oscillation | select oscillation mode (rest duty 50 %), go to rest, **enable the motor** |
| `c<duty>` | both | constant duty in %, decimals accepted: `c50`, `c60.5` |
| `o<mean>,<amp>,<freq>` | oscillation only | sine on the duty: `o50,20,0.5` = 50 % $\pm$20 % at 0.5 Hz; the Arduino builds the sine at 100 Hz, starting at the mean going up |
| `x` | both | back to the rest duty of the current mode |
| `e` | both | enable the ESCON (enable line high) |
| `d` | both | rest duty, then disable the ESCON: motor coasts, *no braking* (send `x` and wait first for a controlled stop) |

- **Safe start (new):** at power-up or reset the motor is *disabled* (switch `START_ENABLED 0`) and the default mode is oscillation (bipolar, current `.edc`). Before, the Arduino booted in constant mode with the motor enabled: with a bipolar `.edc` the 10 % rest duty means full negative speed until a mode was sent.
- Switches at the top of the file: `DEFAULT_MODE`, `START_ENABLED`, `REST_DUTY_CONST` (10), `REST_DUTY_OSC` (50), `USE_LORA` (0, **untested**).
- The Arduino **resets** every time it is re-plugged or its USB port is re-opened: send the mode again at each session (the session script does it).
- Telemetry: 4 bytes every 250 ms (details in Section 1.8).

## 1.3 Phone: plug the Arduino, Gobannos, hotspot

Phone

1. Hotspot on, Gobannos open in the foreground; note the IP it shows (Section 1.4).
2. Plug the Arduino into the phone and accept the USB permission for **Gobannos**. Do not mix with a Serial USB terminal app.

## 1.4 Laptop: connection and IP settings

VS Code PowerShell

- Connect the laptop to the phone's hotspot.
- `connect_v3.py`: `IP_PREFIX`, `NETWORK`, `DEFAULT_PHONELIST` (with `IP_PREFIX = "10"`, phone 36 is `10.153.247.36`). The session asks for phone ID and network number (defaults from this file) and gives the resulting IP to every script it launches. `py connect_v3.py` prints the configured addresses and scans the subnet with `nmap` (must be installed).
- Python packages: `numpy pandas matplotlib` (`pyserial` only for LoRa).

To check the laptop/phone connection:

PowerShell (laptop)

```
ping <phone-ip>
Test-NetConnection -ComputerName <phone-ip> -Port 8080
```

*Expected result:* 4 successful replies with `0% loss`, and `TcpTestSucceeded : True`. If requests time out, confirm your laptop is connected to the phone's active hotspot, and that Gobannos is open in the foreground.

Browser check: `http://<phone-ip>:8080/status` should show `STOPPED` (or a running state).

Clone repository with all my scripts: https://github.com/Turbotice/icewave/tree/main/icewave/chipiron

**Install phonefleet** This is no longer needed for v3 of my scripts, but it is required for termux/ssh debugging.

1. Clone `phonefleet` (https://github.com/Turbotice/phonefleet) in VS Code.
2. In the terminal, in the folder that contains `setup.py`: `pip install -e .`
3. Check: `py -c "import phonefleet.server.connect as c; print(c.__file__)"`. If you get `ModuleNotFoundError`, the likely causes are a different Python environment than the one used for `pip install -e .`, or missing `__init__.py` files in the `phonefleet/` subfolders.

**Find the phone's IP.** Any of these:

- **Gobannos app (easiest)**: it displays the IP when started.
- **Termux** (needs `pkg install termux-api` and the Termux:API app): `termux-wifi-connectioninfo`; the `"ip"` field is the phone's address. This worked.
- **Laptop**, with the hotspot connected: `ipconfig` (PowerShell), then in the Wi-Fi section the **default gateway** ("Passerelle par défaut") is the phone's IP.

**USB access.** In Gobannos, if the **USB STATUS** line first shows *"demande de permission USB non envoyée"*, the phone cannot talk to the Arduino (`/usb-cmd/...`).

1. Plug the Arduino into the phone (USB cable / OTG adapter).
2. Accept the Android permission pop-up for the USB device and choose Gobannos.
3. Check in Gobannos that the USB status changed, and send some URL commands to see that communication is established (Section 3).

## 1.5 Which test?

| Test | Arduino mode | ESCON mapping | Script run by the session | Rest duty |
|---|---|---|---|---|
| Oscillation, firmware mode (smooth sine) | `mo` | bipolar | `chipiron_oscillate_v3.py -mode firmware` | 50 % |
| Oscillation, stream mode (coarse, not recommended) | `mo` or `mc` | bipolar or unipolar | `chipiron_oscillate_v3.py -mode stream` | from `.edc` |

| Test | Arduino mode | ESCON mapping | Script run by the session | Rest duty |
|---|---|---|---|---|
| Constant speed, ramps, steps, up/down cycles, ramp-planner CSV | `mo` (bipolar) or `mc` (unipolar) | bipolar or unipolar | `chipiron_run_v3.py` | from `.edc` |
| Manual (URL/keyboard commands, any other script) | any | any | none or your script | from `.edc` |

**Motor mapping and test types**

The rest duty of the Arduino mode (`mc` 10 %, `mo` 50 %) must give 0 rpm with the `.edc` loaded in ESCON Studio. `chipiron_session_v3.py` stops with an ERROR if they do not match, and refuses firmware-mode oscillation with `mc` or with a unipolar `.edc`.

## 1.6 Run the session: `chipiron_session_v3.py`

VS Code terminal, with `connect_v3.py`, `run_gobannos_v3.py`, `escon_edc_v3.py`, `chipiron_helpers_v3.py`, `chipiron_run_v3.py` and `chipiron_oscillate_v3.py` next to the script (`mobile_v3.py` is not needed, only optional).

### 1.6.1 Launch and sequence

Terminal (VS Code)

```
py chipiron_session_v3.py                                  (asks the mode)
py chipiron_session_v3.py save_mode                        (no experiment: ls, save
    ↪ )
py chipiron_session_v3.py full_session                     (asks Arduino mode, .edc)
py chipiron_session_v3.py full_session mo                  (bipolar: oscill, ramps,
    ↪ steps)
py chipiron_session_v3.py full_session mc                  (unipolar: ramps, steps)
py chipiron_session_v3.py full_session mo parameters_Sep26.edc
py chipiron_session_v3.py full_session skip                (send no mode)
```

The `.edc` can be anywhere on the line; without it the script asks (default = the one used last). `skip` leaves the Arduino as is: after a reset the motor stays *disabled* until you send `e`, `mo` or `mc`.

Sequence of a `full_session`:

1. phone ID and network number → IP; the `.edc` is loaded **once** (summary printed) and handed to every script launched afterwards (environment variables), so none of them asks for it again;
2. status of the phone;
3. Arduino mode `mc`/`mo`/`skip`: checked against the `.edc`, then sent (this also enables the motor at the rest duty);
4. choice of experiment and **all its questions**, asked *before* the acquisition starts (errors such as a duty out of range stop the session here, nothing has moved);

5. 0 rpm command (`c<rest duty>` computed from the `.edc`: `c50.0` bipolar, `c10.0` unipolar), then a wait of `ZERO_WAIT_S` = 30 s at rest;
6. `/start` (acquisition on the phone), experiment runs (scripts are called with `-no-record`, so they do not call `/start` and `/stop`);
7. motor sent back to rest in any case (also on Ctrl-C), `/stop`, `ls`, save.

### 1.6.2 Choice 1: manual

The acquisition starts, then the script waits. Type at the > prompt: `e` (enable), `d` (disable), `x` (rest), `c60.0` (duty), `o50,20,0.2` (sine, `mo` only); **Enter stops** the acquisition. The motor is sent back to rest afterwards. Useful to test the chain, or for the A1/A2 identification (Section 3).

### 1.6.3 Choice 2: oscillation (firmware or stream)

| Prompt | Example | Meaning |
|---|---|---|
| Oscillation mode | `firmware` | `firmware`: one message `o<mean>,<amp>,<freq>`, the Arduino builds the sine at 100 Hz (needs `mo` and a bipolar `.edc`); default when these hold. `stream`: Python sends a new `c<duty>` every `-dt` s, works with anything but is coarse and **not recommended** (steps of 0.1 % duty, only for periods $\gtrsim 20$ s, less smooth sines) |
| Mean speed (rpm) | 0 | `-mean-rpm`; 0 = oscillate around standstill (bipolar); default 400 if unipolar |
| Amplitude (rpm) | 100 | `-amp-rpm`; below about 80 rpm ($\pm 4$ % duty) the sign did not reverse on the lab motor (Section 6.2) |
| Frequency (Hz) | 0.1 | `-freq` |
| Number of cycles | 5 | `-cycles`; duration = cycles / frequency (50 s here) |
| Extra arguments | Enter | optional: `-song`, `-dt 0.5` (stream), `-verbose` |

The session already checks (before starting): mean $\pm$ amplitude must fit in 10–90 % duty, and the *peak acceleration* of the sine, amp $\cdot\, 2\pi f$, is compared with the ESCON speed ramp (2000 rpm/s here): above it the limiter flattens the sine, a warning gives the largest usable amplitude and asks for confirmation. The rpm are converted with the `.edc` mapping (duty amplitude = amp / 20 rpm per %). With these values, `chipiron_oscillate_v3.py` sends one message, `o50.00,5.00,0.1000`, and the firmware builds the sine. The script can also be run alone (`-no-record` when the acquisition is handled elsewhere; `-yes` skips the confirmation):

Terminal (VS Code): oscillation examples

```
py chipiron_oscillate_v3.py --edc parameters_Sep26.edc --mean-rpm 0 --amp-rpm 100 --
    ↪ freq 0.1 --cycles 5
py chipiron_oscillate_v3.py --mean-rpm 0 --amp-rpm 300 --freq 0.5 --cycles 10 --no-
    ↪ record --yes
py chipiron_oscillate_v3.py --mode stream --dt 1 --mean-rpm 0 --amp-rpm 100 --freq
    ↪ 0.02 --cycles 3 --no-record
```

```
py chipiron_oscillate_v3.py --rpm-at-10 -800 --rpm-at-90 800 --mean-rpm 0 --amp-rpm
    ↪ 200 --freq 0.2
py chipiron_oscillate_v3.py --link lora --serial-port COM5 --mean-rpm 400 --amp-rpm
    ↪ 100 --freq 0.1
```

**Quick estimate of the acceleration (ramp) needed**

For a commanded sine $v(t) = A\sin(2\pi ft)$; maximum acceleration is $a_{\max} = A\,2\pi f$. NB: $rad/s \to rpm$: multiply by $\frac{60}{2\pi} \approx 10$. This must stay below the ESCON speed ramp (2000 rpm/s here, 10 rpm/s in the slow `.edc`!), otherwise the limiter flattens the sine. Conversely, the largest usable amplitude at frequency $f$ is $A_{\max} = a_{\text{ramp}}/(2\pi f)$. The ESCON *speed controller gain* is a second, independent limit: with a gain of 1 the loop is a low-pass filter with $\tau \simeq 4.6$ s and a 0.1 Hz sine comes out with a third of its amplitude (Section 4).

| `chipiron_oscillate_v3.py` argument | Default | Meaning |
|---|---|---|
| `-mean-rpm`, `-amp-rpm`, `-freq` | required | sine in rpm and Hz |
| `-cycles` | 5 | number of periods |
| `-mode` | `firmware` | `firmware` (needs `PWM_full_v3.ino` in `mo`) or `stream` |
| `-dt` | 1.0 | stream mode: seconds between two commands |
| `-edc FILE` | asked | ESCON file (ignored when called from the session) |
| `-rpm-at-10`, `-rpm-at-90` | none | manual mapping instead of the `.edc`: give *both* (e.g. $-800$ and 800) |
| `-link` | `http` | `http` through the phone, or `lora` (**untested**) |
| `-phone-ip IP` | env / default | phone IP (the session sets it) |
| `-serial-port PORT` | none | LoRa bridge port (with `-link lora`) |
| `-no-record` | off | do not call `/start` and `/stop` |
| `-yes` | off | no confirmation if the ESCON ramp is too slow (added by the session) |
| `-song` | off | play `test.mp3` (needs the `play` command) |
| `-verbose` | off | print every HTTP request |

At the end (also on Ctrl-C or error) the motor goes back to rest: `x` (firmware) or `c<rest duty>` (stream).

### 1.6.4 Choice 3: run a program (constant speed, ramps, steps, cycles, CSV)

`chipiron_run_v3.py` builds a *program*, i.e. a list of (duration, duty) commands, checks it against the ESCON ramp limiter, optionally saves it as CSV, and replays it with one `c<duty>` per row at the right time. In the session it runs in **two steps**:

1. *Step A (before the acquisition)*: `py chipiron_run_v3.py -build-only programs/session_<date>.csv`: you answer the questions (mode, blocks, rest before/after); nothing is sent to the motor. Or answer the first question with an **existing CSV** (a program saved earlier or a ramp-planner CSV) and nothing is built.
2. *Step B (during the acquisition)*: `py chipiron_run_v3.py -csv <file> -yes -no-record`: replays it without any question. It refuses to run if the CSV was planned with another mapping than the current `.edc`.

| chipiron_run_v3.py argument | Default | Meaning |
|---|---|---|
| -mode manual\|load_ramp_planner | asked | build by hand, or start from a CSV (default if -csv) |
| -csv FILE | none | ramp-planner CSV or saved program |
| -yes | off | with -csv: run immediately, no question |
| -build-only OUT.csv | none | build, save to OUT.csv, exit (no motor command) |
| -edc FILE | asked | ESCON file (ignored from the session) |
| -dt S | 1.0 | seconds between two commands inside a ramp (a ramp is a staircase of small steps) |
| -rest-before S, -rest-after S | asked (30, 0) | rest at 0 rpm added before/after the program (baseline in the recording) |
| -link, -phone-ip, -serial-port | http | as for oscillation |
| -no-record | off | do not call /start and /stop |
| -no-reset | off | do not send the rest command at the end |
| -s, -song | off | play a sound |
| -verbose | off | print every HTTP request |

Terminal (VS Code): run examples

```
py chipiron_run_v3.py                                                    (everything
    ↪ interactive)
py chipiron_run_v3.py --edc parameters_Sep26.edc --no-record       (build, save, run
    ↪ now?)
py chipiron_run_v3.py --build-only programs/test1.csv --rest-before 30 --rest-after
    ↪ 10
py chipiron_run_v3.py --csv programs/test1.csv --yes               (run a saved
    ↪ program)
py chipiron_run_v3.py --mode load_ramp_planner --csv ramps/myramp.csv   (load, add
    ↪ blocks, run)
py chipiron_run_v3.py --csv programs/test1.csv --yes --link lora --serial-port COM5
```

**Blocks of the manual mode** (menu; values in rpm or %, u switches the unit; every block can be repeated $N$ times with an optional pause at rest; every value is checked against 10–90 % duty):

| Menu | Block | Questions |
|---|---|---|
| 1 | hold | value, duration |
| 2 | ramp | from, to, by *duration* (d) or by *rate* (r, in rpm/s or %/s); warns if shorter than the ESCON ramp allows (minimum time printed) |
| 3 | staircase | plateau values: 1 typed list, 2 linear (first, last, number), 3 logarithmic (positive first/last, number); optional mirror staircase (comes back down); same time on every plateau or one by one; optional rest at 0 between plateaus |
| 4 | up/down cycle(s) | A, B, ramp A→B (duration or rate), hold at B, ramp B→A, hold at A, number of cycles |
| 5 | repeat whole program | total number of runs, pause at rest between runs |
| 6 | load CSV | a ramp-planner CSV or saved program, appended (mapping checked) |
| 7 | remove last block | |

| Menu | Block | Questions |
|---|---|---|
| u | switch unit | rpm ↔ % (only % if the set value is not a speed) |
| 9 | done | rest before/after, FINAL PROGRAM summary with ESCON warnings, "Save as CSV?", "Run it now?" |

**Ramp planner** (`chipiron_ramp_planner_v3.py`): plans a staircase of $N$ plateaus (linear or logarithmic) in rpm, rad/s or %, prints the ramp (requested value, duty requested, duty sent, speed really obtained with the Arduino resolution, error), warns about rounding that merges points, plateaus the ESCON ramp cannot reach, or duty out of range, and saves a CSV for `chipiron_run_v3.py`.

Terminal (VS Code): ramp planner

```
py chipiron_ramp_planner_v3.py
py chipiron_ramp_planner_v3.py --edc parameters_Sep26.edc --unit rpm --type log --
    ↪ start 10 --end 500 --points 12 --time 60 --folder ramps
py chipiron_ramp_planner_v3.py --unit rad/s --type linear --start 1 --end 40 --points
    ↪  10 --time 30
py chipiron_ramp_planner_v3.py --unit % --start 52 --end 80 --points 8 --time 20 --no-
    ↪ save
```

Arguments (all asked if omitted): `-edc`, `-unit` (`rpm`, `rad/s` or `%`), `-type linear|log` (log needs values $> 0$), `-start`, `-end`, `-points` ($\geq 2$), `-time` (seconds per plateau), `-folder` (giving it saves without asking, default file name), `-name`, `-no-save`. The CSV file name contains the parameters (e.g. `ramp_log_10to500rpm_12pts_60s_bipolar_...csv`); the header (comment lines starting with a hash sign, "key: value") stores the mapping so that `chipiron_run_v3.py` can refuse a CSV planned with another `.edc`; columns: `t_start_s, duration_s, duty_pct, rpm, omega_rad_s, kind, segment`. Then run it with `py chipiron_run_v3.py -csv ramps/<file>.csv -yes`, or in the session (choice 3, existing CSV).

### 1.6.5 Choice 4: another script

Asks the script name and its arguments; `-no-record` is added for the scripts that accept it (`chipiron_run_v3.py`, `chipiron_oscillate_v3.py`). The `.edc` configuration and the phone IP are available to the script through the environment variables `CHIPIRON_MOTOR_CFG` and `CHIPIRON_PHONE_IP` (see `chipiron_helpers_v3.py`).

### 1.6.6 Running a script without the session

Run and oscillate scripts can be used alone (e.g. with the acquisition started from `mobile_v3.py`): they then ask for the `.edc` (or `-edc`), use the phone IP `-phone-ip` > `CHIPIRON_PHONE_IP` > default `10.153.247.36`, and call `/start` and `/stop` themselves unless `-no-record`. **Standalone they do not send the rest command at the beginning**: send `c50.0` (or `mo`) yourself first.

## 1.7 Stop, list and save the files

VS Code (last questions of the same session).

| Prompt | Example | Meaning |
|---|---|---|
| Choice (save) | 4 | 1 one file, 2 all, 3 one date, 4 one date and time |
| Root folder on laptop | Enter | default `Phonefleet_data` |
| Date | Enter | default today, `YYYY-MM-DD` |
| Time | `08:45:19` | read in the file names printed by `ls` (`...T08_45_19...`) |

Result

```
Phonefleet_data/
  2026-10-15/
    08-45-19/
      2026-10-15T08_45_19-Usb-...csv
      2026-10-15T08_45_19-android_sensor_accelerometer-...csv
      2026-10-15T08_45_19-android_sensor_gyroscope-...csv
      2026-10-15T08_45_19-android_sensor_magnetic_field-...csv
      2026-10-15T08_45_19-gps-...csv
```

Later, to fetch files without any experiment (`ls` then save only): `py chipiron_session_v3.py save_mode`.

**Alternative:** `mobile_v3.py` is a utility menu to manage the phone and pull files to the laptop (files named `P<phone>_D<original name>` in `Phonefleet/`). Commands at the > prompt: `network`, `phones` (e.g. 36 or 36:38), `status`, `start`, `stop`, `ls <phone_id>`, `pull <phone_id> <index>` or `pull <phone_id> <first:last>`, `usb <command>` (e.g. `usb mo`, `usb c60.0`, `usb x`, `usb d`; sent to *all* active phones), `time`, `exit`.

**Optional: clock synchronisation.** `mobile_v3.py` command `time`, or `py tsync_v3.py -phone 36 -n 100 -iter 3 -folder Phonefleet/Gobannos_Tsync`, measures the lag laptop–phone over UDP (ports 5000/5001) and saves it as csv. Not needed for a basic test; useful to relate laptop times to phone timestamps.

## 1.8 Decode and plot: `chipiron_plotting_v3.py`

VS Code

### 1.8.1 How the Arduino data reach the files and are decoded

Every 250 ms the Arduino reads the two analog inputs (10-bit ADC, 0–5 V, A1 on pin A1, A2 on pin A2) and sends **4 bytes** over USB:

| byte | content | time in the cycle | phone rows |
|---|---|---|---|
| $b_1$ | ADC(A1) & 0xFF (low 8 bits) | 0 ms | one row each, |
| $b_2$ | ADC(A1) $\gg 4$ | $\approx 10$ ms | timestamp in |
| $b_3$ | ADC(A2) & 0xFF | $\approx 50$ ms | microseconds, |
| $b_4$ | ADC(A2) $\gg 4$ | $\approx 60$ ms | signed int8 |

Gobannos logs *each byte as one row* of the `Usb` csv (timestamp in $\mu$s on the phone clock, byte as signed int8, e.g. $-43 = 213$). Decoding (`chipiron_decode_v3.py`): adc $= (b_2 \ll 4) \,|\, (b_1 \,\&\, \texttt{0x0F})$,

with the consistency check $(b_1 \gg 4) = (b_2 \,\&\, \texttt{0x0F})$ (a failed check gives NaN). The start of a cycle is found from the **time gaps** ($> 100$ ms between cycles), not from "every 4th row", so a lost byte only loses its own cycle. Cycles with 4 rows are `full`; cycles with only 2 rows $\approx 50$ ms apart (the phone lost $b_2$ and $b_4$) are `low8`: only the low 8 bits are known, so `A1_lo`/`A2_lo` are filled and the ADC counts stay NaN (values are aliased); anything else is dropped and counted in the printed summary.

Conversion to physical units uses the ESCON analog outputs (AO shift of the current `.edc`): $V = \text{adc} \cdot V_{\text{ref}}/1023$ with $V_{\text{ref}} = 4.88$ V (**measured**, see the box below). **A1 is wired to AO1 = actual current, A2 to AO2 = actual speed** (lab test of 20/10/2026): current $= (V - 2)/0.4$ A (1 count $\approx 0.012$ A); speed $= (V - 2)/0.002$ rpm (1 count $\approx 2.4$ rpm).

**Measure $V_{\text{ref}}$ (ADC reference)**

The ADC converts $0 \ldots V_{\text{ref}}$ into $0 \ldots 1023$. $V_{\text{ref}}$ is *not* exactly 5 V: it is the real voltage of the Arduino 5 V pin, which depends on how the board is powered (USB from a laptop, from the phone…). Measure it with a multimeter between **GND and 5V** of the Arduino, powered *as in the experiment*, and write it in `ADC_VREF` (`chipiron_decode_v3.py` and `chipiron_plotting_v3.py`). Measured on the lab Arduino: **4.88 V** (using 5 V instead gave a $\sim 2.4\,\%$ error on every voltage, speed and current). **Re-measure on the Arduino of Chipiron.**

**Columns** (the csv files have no header; names are assigned by the scripts):

| File | Columns | Meaning |
|---|---|---|
| Usb (raw rows) | `t_us, usb_value, byte, t_s` | timestamp ($\mu$s), signed byte, byte 0–255, seconds since first row |
| Usb (decoded) | `t_us, kind, A1_lo, A2_lo, A1_adc, A2_adc, t_s, A1_V, A2_V` | one row per 250 ms cycle; `kind` = `full` or `low8` |
| Usb + `.edc` | `A1_current_A, A2_speed_rpm` | added when an `.edc` is given (A1 = AO1 current, A2 = AO2 speed) |
| accelerometer | `t_us, ax, ay, az, t_s` | Android convention, m/s$^2$ |
| gyroscope | `t_us, gx, gy, gz, t_s` | rad/s |
| magnetic_field | `t_us, mx, my, mz, t_s` | $\mu$T |
| gps | `t_us, gps_1, gps_2, gps_3, t_s` | meaning unknown, empty so far |

Rows with timestamp $\leq 0$ are placeholders (the phone pre-allocates its buffers) and are removed; $t_s$ = seconds since the first sample of the file, so different files are *not* on a common time axis.

### 1.8.2 Use

Terminal (VS Code), or run the cells one by one

```
py chipiron_plotting_v3.py
```

The first cell asks for the folder with the csv files of one acquisition (e.g. `Phonefleet_data/2026-10-15/08-45-19`) and lists them. Then one cell per file (`Usb` first), each printing the rows read/removed and the first five lines, then asking:

| Prompt | Example | Meaning |
|---|---|---|
| Folder | path | folder of one date/time |
| Usb data | `1` | 1 decoded ADC channels A1/A2, 2 raw rows as logged, 3 the 4 bytes by role ($b_1 \dots b_4$) — see below |
| Units (option 1) | `1` | 1 ADC counts (decoded, not converted), 2 volts at the pin, 3 physical units (rpm, A) with an `.edc` |
| Decoded: which (option 1) | `3` | 1 A1, 2 A2, 3 both (two figures) |
| Raw rows: which (option 2) | `4` | 1 all, 2 even rows, 3 odd rows, 4 even and odd (two figures) |
| Bytes: which (option 3) | `5` | 1 $b_1$, 2 $b_2$, 3 $b_3$, 4 $b_4$, 5 all four (four figures) |
| Column to plot (y), as a function of (x) | `A1_adc`, `t_s` | index or name |
| x and y axis labels | Enter or text | default = column name |
| Scale | `1` | 1 linear, 2 semilogx, 3 semilogy, 4 loglog |
| Save plots? | `all` | `none`, `all`, numbers (`1,3`) or ranges (`2-4`) |

Figures stay open while the script goes on. Saved figures go to `<data folder>/figures/`, named `date_time_sensor_y_vs_x_scale.png`; an existing file is never overwritten.

**The three Usb plotting options.**

1. **Decoded** (the one to use for measurements). The bytes are rebuilt into ADC counts (Section 1.8.3), **one point per 250 ms cycle**. You then choose the unit: ADC counts (safest: nothing assumed about $V_{\text{ref}}$ or the `.edc`), volts (uses `ADC_VREF`), or rpm/A (needs the `.edc` loaded in ESCON Studio during the recording; A1 = current, A2 = speed). If only low bytes were logged, it plots `A1_lo`/`A2_lo` (aliased).
2. **Raw rows as logged**. No decoding: one point per byte, i.e. 4 points per cycle, value = signed byte as written by Gobannos. "Even"/"odd" only mean the *row* parity, which does not match the byte roles if the recording started mid-cycle (see the trap above). Use it to check the file, not to measure.
3. **Bytes by role**. The 4 bytes $b_1 \dots b_4$ taken from the *decoded cycles* (so using time gaps, not parity): $b_1, b_3$ = low bytes of A1/A2, $b_2, b_4$ = adc $\gg 4$. Useful to debug the decoding (a noisy $b_1$ is normal; $b_2$ and $b_4$ should vary slowly).

To decode in your own script:

Python

```
from chipiron_decode_v3 import read_raw_usb
from escon_edc_v3 import load_edc
d = read_raw_usb("P36_D2026-10-15T08_45_19-Usb-1-....csv")      # decoded, one row
    ↪ per cycle
load_edc("parameters_Sep26.edc").apply_to_frame(d)              # default wiring: A1
    ↪ = AO1 (current), A2 = AO2 (speed)
raw = read_raw_usb("....csv", decode=False)                     # rows exactly as
    ↪ logged
```

**A1/A2: which is speed?**

Lab test (motor held at 600 rpm in ESCON Studio, compared with the Arduino readings, 20/10/2026): **A1 ← AO1 = current, A2 ← AO2 = speed**. This is now the default `wiring` in `apply_to_frame` and in the plotting script (`WIRING`). The older comments of the firmware (A1 = speed) were wrong for the lab set-up; in `PWM_full_v3.ino` the variables `telemSpeed` / `telemCurrent` were *not* renamed and are swapped (`telemSpeed` holds A1 = current, `telemCurrent` holds A2 = speed). The wiring is *not detected*: it depends on which function is set on AO1/AO2 in the `.edc` and on the cables; **re-check on Chipiron** (Section 3.2). Still **to confirm**: the timestamp clock and the four GPS columns.

### 1.8.3 Coding and decoding explained

**The problem.** The Arduino measures each analog input with a 10-bit ADC: an integer `adc` between 0 and 1023. But the USB serial link only sends **bytes** (8 bits, values 0–255). 1023 does not fit in one byte, so each measurement is sent as **two bytes**. $b_1, \ldots, b_4$ are just the names of the 4 bytes sent in one telemetry cycle (every 250 ms):

| byte | what it is | formula in the firmware | sent at |
|---|---|---|---|
| $b_1$ | A1, *low* byte | $\text{adc}_1$ & 0xFF (bits 0–7) | 0 ms |
| $b_2$ | A1, *high* part | $\text{adc}_1 \gg 4$ (bits 4–9, value 0–63) | $\approx 10$ ms |
| $b_3$ | A2, *low* byte | $\text{adc}_2$ & 0xFF | $\approx 50$ ms |
| $b_4$ | A2, *high* part | $\text{adc}_2 \gg 4$ | $\approx 60$ ms |

(& 0xFF keeps the 8 lowest bits; $\gg 4$ shifts the number 4 bits to the right, i.e. divides by 16 and drops the remainder.)

**Worked example** (adc $= 700$, binary 10 1011 1100):

| | |
|---|---|
| low byte $b_1 = 700$ & $255$ | $188 =$ 1011 1100 |
| high part $b_2 = 700 \gg 4$ | $43 =$ 10 1011 |
| rebuild: $(b_2 \ll 4) \,\vert\, (b_1$ & 0x0F$)$ | $43 \cdot 16 + 12 = 688 + 12 = 700$ ✓ |

**Why do bits overlap?** $b_1$ already contains bits 0–7 and $b_2$ contains bits 4–9, so bits 4–7 are sent *twice*: they are the upper half of $b_1$ and the lower half of $b_2$. We therefore only need the lowest 4 bits of $b_1$ (`b1 & 0x0F`) to rebuild the number, and the overlap is a free **consistency check**: $(b_1 \gg 4)$ must equal $(b_2$ & 0x0F$)$. Here $188 \gg 4 = 11$ and $43$ & $15 = 11$ ✓. If they differ, a byte was lost or corrupted and the decoder returns NaN for that value.

**What ends up in the file.** Gobannos writes *each byte as one row* of the `Usb` csv: a timestamp ($\mu$s, phone clock) and the byte as a *signed* int8, so 188 is logged as $-68$ (the decoder takes `byte & 0xFF` to get 188 back). One cycle = 4 consecutive rows.

**The odd/even trap.** If the recording starts exactly at the beginning of a cycle, the rows are $b_1, b_2, b_3, b_4, b_1, \ldots$: *even* rows (0, 2, 4…) are the low bytes $b_1, b_3$ and *odd* rows are $b_2, b_4$. But if the phone starts logging in the middle of a cycle, or loses a byte, everything shifts by one row and "even" rows become $b_2, b_4$. The row parity is therefore **not** a reliable role. The decoder uses the **time gaps** instead: inside a cycle the gaps are $b_1 \to b_2 \approx 10$ ms, $b_2 \to b_3 \approx 40$ ms, $b_3 \to b_4 \approx 10$ ms, and between two cycles $b_4 \to b_1 \approx 190$ ms. A gap $> 100$ ms means a new cycle starts, and the first byte after it is $b_1$. A lost byte then only ruins its own cycle. Cycles are classified as:

- `full`: 4 rows → ADC counts rebuilt (NaN where the check fails);
- `low8`: 2 rows $\approx 50$ ms apart (only $b_1, b_3$ logged, $b_2, b_4$ lost): only the low 8 bits are known, so the value is known modulo 256 (aliased) and `A1_adc`/`A2_adc` stay NaN;
- anything else: dropped and counted in the printed summary.

**Summary of the chain:**

voltage on pin $\xrightarrow{\text{ADC}}$ adc $\in [0, 1023]$ $\xrightarrow{\text{split}}$ $(b_1, b_2)$ $\xrightarrow{\text{USB, one row per byte}}$ csv $\xrightarrow{\text{decode}}$ adc $\xrightarrow{\cdot V_{\text{ref}}/1023}$ $V$ $\xrightarrow{\text{.edc}}$ rpm

## 1.9 End of the test and diagnosis

- Motor at rest on the ESCON display, no error; phone status `STOPPED` (printed by the session). If the motor must be left for a long time, send `d` (disable) or switch the supply off.
- Keep the program CSV (`programs/session_<date>.csv`) next to the data: the command times are not saved with the phone files (Section 5).
- Scripts that were modified are pushed to git; ESCON parameters saved and labelled.
- **Termux** Diagnosis only: ssh to the phone (Section 1.11). From the laptop, `py connect_v3.py` prints the configured addresses and scans the subnet with `nmap`.

## 1.10 Scripts at a glance

| Script | Task and purpose | Example command | Asks / takes |
|---|---|---|---|
| `escon_edc_v3.py` | reads an ESCON `.edc`: mapping, rest duty, ramp, motor type, analog-output conversion; module used by all others | `py escon_edc_v3.py a.edc -no-save` | `.edc`, save folder |
| `chipiron_patch_edc_v3.py` | makes a patched copy of an `.edc` (AO shift, bipolar mapping, ramp, motor type, offset) | `py chipiron_patch_edc_v3.py -in a.edc -out b.edc -bipolar-setpoint 800 -ramp 2000` | arguments only |
| `PWM_full_v3.ino` | Arduino firmware: PWM + enable, modes `mc`/`mo`, sine, telemetry; safe start | flashed from the Arduino IDE; commands `mo`, `c60.0`, `o50,5,0.1`, `x`, `e`, `d` | text commands |
| `chipiron_session_v3.py` | full measurement: mode, 0 rpm, record, experiment, rest, stop, save | `py chipiron_session_v3.py full_session mo a.edc` | mode, phone, experiment, save |
| `chipiron_run_v3.py` | builds/replays programs: holds, ramps, staircases, cycles, CSV | `py chipiron_run_v3.py -csv programs/p.csv -yes -no-record` | blocks or CSV |
| `chipiron_oscillate_v3.py` | sinusoidal speed, firmware or stream mode | `py chipiron_oscillate_v3.py -mean-rpm 0 -amp-rpm 100 -freq 0.1 -no-record` | rpm, Hz, cycles |

| Script | Task and purpose | Example command | Asks / takes |
|---|---|---|---|
| chipiron_ramp_planner_v3.py | plans linear/log staircases in rpm, rad/s, %, saves CSV | py chipiron_ramp_planner_v3.py -unit rpm -type log -start 10 -end 500 -points 12 -time 60 -folder ramps | unit, range, points, time |
| chipiron_helpers_v3.py | shared module: config from .edc, HTTP/LoRa links, ramp-limiter check, CSV; not run directly | import chipiron_helpers_v3 as cm | none |
| chipiron_decode_v3.py | decodes the Usb csv into ADC and volts | read_raw_usb(path) (Python) | file |
| chipiron_plotting_v3.py | plots one acquisition, optional rpm/A conversion | py chipiron_plotting_v3.py | folder, columns, scale |
| mobile_v3.py | manual menu: status, start/stop, ls, pull, usb, time | py mobile_v3.py then usb mo | menu commands |
| connect_v3.py | phone ID → IP, nmap scan | py connect_v3.py | none |
| run_gobannos_v3.py | HTTP calls to the phone (module) | gob.usb_cmd(ip, "mo") (Python) | none |
| tsync_v3.py, rw_data_v3.py | clock sync laptop–phone, csv writer | py tsync_v3.py -phone 36 | phone, count |

## 1.11 Optional (not fully tested): SSH and Termux

**What I've tested:** Set up SSH access from a Windows laptop to Termux on the Fairphone, browse files stored on the phone, locate previously recorded USB serial measurements, and transfer those measurements to the laptop over the local Wi-Fi/hotspot connection.

### 1. Enable SSH access in Termux

Open Termux on the Fairphone. Install and update the required packages:

Termux

```
pkg update
pkg install openssh
```

Set a password for the Termux user (PMMH default):

Termux

```
passwd
```

Find the Termux username:

Termux

```
whoami
```

(something like `u0_a235`).

Start the SSH server:

Termux

```
sshd
```

Termux's OpenSSH server normally listens on port 8022, rather than the standard SSH port 22. Check whether the server process is running with:

Termux

```
pgrep -a sshd
```

### 2. SSH connect from Windows

Connect the Windows laptop to the Fairphone's hotspot, or otherwise ensure both devices can communicate over the same local network, and get the phone's IP. Connect opening a PowerShell (example with test values):

PowerShell

```
ssh -p 8022 u0_a235@10.14.43.89
```

Enter the password created with `passwd` in Termux. The first connection may display a host-authenticity prompt; verify that the address is the expected phone and accept the host key to continue. The terminal may report that the host was permanently added to the list of known hosts.

Confirm the account and home directory:

Termux (over SSH)

```
whoami
pwd
ls -la
```

The observed results were `u0_a235` and `/data/data/com.termux/files/home`.

### 3. Understand the phone's storage paths

From the Termux home directory, inspect the shared Android storage:

Termux (over SSH)

```
cd ~/storage
ls
cd shared
ls
```

The available shared-storage directories included `Download`, `Documents`, `DCIM`, `Pictures`, `Recordings`, and `RotorlaboData`.

Android shared internal storage is accessible through Termux's storage links. The path to the shared storage root is `~/storage/shared`. The actual location of the shared `Download` directory is therefore `~/storage/shared/Download`.

If storage access has not been configured, run:

Termux

```
termux-setup-storage
```

Grant the requested Android storage permission, if prompted, and retry the directory listing. This provides access to permitted shared storage; it does not automatically grant access to private files belonging to other Android apps.

Search for potentially relevant files in Termux's home directory:

Termux (over SSH)

```
find ~ -type f \( \
  -iname '*serial*' -o \
  -iname '*rotor*' -o \
  -iname '*.csv' -o \
  -iname '*.txt' -o \
  -iname '*.log' \
\) 2>/dev/null | head -200
```

### 4. Exit and transfer files

Exit the SSH session if necessary by entering:

Termux (over SSH)

```
exit
```

Then in a Windows PowerShell (example folder to copy):

PowerShell (laptop)

```
scp -P 8022 -r u0_a235@10.14.43.89:storage/shared/Download/Matilde "C:\Users\matil\
    ↪ Downloads\"
```

Enter the Termux password when prompted. Details:

- `-P 8022`: connect to the phone's SSH server on port 8022.
- `-r`: recursively copy the directory and its contents.
- `u0_a235@10.14.43.89`: the Termux account and phone IP address.
- `storage/shared/Download/Matilde`: the source directory, relative to the Termux home directory.
- `C:\Users \matil \Downloads` : the local destination on Windows (with a trailing backslash in the command).

Verify the transfer in Windows PowerShell:

PowerShell (laptop)

```
Get-ChildItem "C:\Users\matil\Downloads\Matilde"
```

The transferred file should appear in the listing.

To copy only one specific file, same idea, for instance:

PowerShell (laptop)

```
scp -P 8022 u0_a235@10.14.43.89:storage/shared/Download/Matilde/
    ↪ serial_20261009_151212.txt "C:\Users\matil\Downloads\serial_20261009_151212.
    ↪ txt"
```

# 2 Ways to send an input

## 2.1 Command grammar (what the Arduino understands)

| Command | Mode | Example | Result with the current bipolar `.edc` (20 rpm per %) |
|---|---|---|---|
| `mc`, `mo` | any | `mo` | select mode, rest duty (10 / 50 %), enable the motor |
| `c<duty>` | both | `c50.0` | 0 rpm |
| | | `c60.0` | $+200$ rpm |
| | | `c40.0` | $-200$ rpm |
| `o<mean>,<amp>,<freq>` | `mo` only | `o50,10,0.1` | sine $0 \pm 200$ rpm, 10 s period; ignored in `mc` |
| `x` | both | `x` | rest duty of the mode |
| `e` / `d` | both | `d` | enable / disable the motor (coasts) |

Duty values are in %, clamped to 10–90 % by the Arduino (and refused by the scripts outside this range); decimals are accepted but the hardware resolution is $100/255 = 0.39$ % (about 7.8 rpm here). With a unipolar `.edc`: 10 % = 0 rpm and $+800$ rpm at 90 %.

## 2.2 Channels

| Channel | Tool / script | Command | Notes |
|---|---|---|---|
| Browser URL | browser, Gobannos server on the phone | `http://<phone-ip>:8080/usb-quick?cmd=c60.0`<br>`http://<phone-ip>:8080/usb-cmd?cmd=o50,20,0.04` | quick test; command typed as it would work |
| Command line | `curl.exe` (PowerShell) or `curl` (Termux) | `curl.exe -v -max-time 5 http://<phone-ip>:8080/usb-quick?cmd=c60.0` | same URL as the browser |
| Manual menu | `mobile_v3.py` | `usb mo`, `usb c60.0`, `usb x`, `usb d` | sent to all active phones |
| Session, manual choice 1 | `chipiron_session_v3.py` | `s`, `d`, `x`, `c60.0`, `o50,20,0.2` at the `>` prompt | acquisition running; motor rest at the end |
| Oscillation, firmware | `chipiron_oscillate_v3.py --mode firmware -mean-rpm 0 -amp-rpm 200 -freq 0.1` | | one message `o...`; smooth sine; `mo` + bipolar |

| Channel | Tool / script | Command | Notes |
|---|---|---|---|
| Oscillation, stream | `chipiron_oscillate_v3.py` | `-mode stream -dt 1 -freq 0.02 ...` | a `c<duty>` every dt; coarse, **not recommended** |
| Constant, ramps, steps, cycles | `chipiron_run_v3.py` | interactive blocks, or `-csv prog.csv -yes` | one `c<duty>` per row at the scheduled time |
| Planned ramp | `chipiron_ramp_planner_v3.py` → run | `-csv ramps/<file>.csv -yes` | staircase, hold per plateau |
| Python | `chipiron_helpers_v3.make_link` or `run_gobannos_v3.usb_cmd` | `link.send("c60.0"); gob.usb_cmd(ip, "x")` | for your own scripts |
| LoRa (**untested**) | `-link lora -serial-port COM5` (run, oscillate) | same commands as text lines | needs the bridge Arduino, `pyserial`, `USE_LORA 1` in the firmware |

# 3 Notes for a first test with Chipiron

## 3.1 Safety and pre-checks

**Before any motor test**

Motor unloaded or free to turn, a hand on the supply switch, small speeds first (a few hundred rpm), current limit set in ESCON Studio (the mechanics will fail well before 9 A, so consider 4–5 A), **battery plugged last**. `d` (disable) makes the motor coast and is not an emergency stop.

1. The `.edc` loaded in ESCON Studio is the one given to the scripts; `py escon_edc_v3.py <file>` shows 50 % → 0 rpm (bipolar) or 10 % → 0 rpm (unipolar).
2. `PWM_full_v3.ino` flashed; Arduino plugged into the phone; Gobannos in the foreground; USB permission accepted.
3. Laptop on the hotspot; `Test-NetConnection -ComputerName <phone-ip> -Port 8080` succeeds.

## 3.2 Identify A1 and A2 (who is speed, who is current)

**A1/A2**

Tested in the lab (20/10/2026): **A1 is current (AO1), A2 is speed (AO2)**; this agrees with the `.edc` (`PARAM_AnalogOutputsConfiguration` = 0x42: AO1 = function 2 = Actual Current, AO2 = function 4 = Actual Speed; the averaged versions are functions 3 and 5). If the function set on an AO or a cable changes, update the `.edc` and `WIRING`. Must be checked on Chipiron, where the wiring may differ (Stéphane says A1 = speed, sent first, A2 = current, sent 50 ms later, then 150 ms before the next loop). Procedure: start the motor manually

(URL or session choice 1), rotate the motor by hand in both directions (the current should not change), then send various c<duty> in both rotation directions (speed should follow the steps, current should spike when the input changes then stay constant), then stop. Alternatively run mobile_v3.py (start, usb ..., stop), then ls <phone> and pull the files, and decode them. Also measure $V_{\mathrm{ref}}$ (Section 1.8.3 box).

## 3.3 Basic tests by URL (no script)

Phone and laptop connected, motor free. Replace <ip> by the phone IP; with a bipolar .edc (one step of 1 % = 20 rpm):

| | Browser |
|---|---|
| http://<ip>:8080/status | STOPPED |
| http://<ip>:8080/usb-cmd/mo | Arduino mode mo, motor armed at rest (0 rpm) |
| http://<ip>:8080/start | acquisition starts |
| http://<ip>:8080/usb-cmd/c55.0 | about +100 rpm |
| http://<ip>:8080/usb-cmd/c45.0 | about -100 rpm |
| http://<ip>:8080/usb-cmd/o50,5,0.1 | sine 0 +/- 100 rpm, 10 s period (mo only) |
| http://<ip>:8080/usb-cmd/x | back to 0 rpm |
| http://<ip>:8080/usb-cmd/d | motor disabled (ESCON shows disabled, motor coasts) |
| http://<ip>:8080/usb-cmd/e | motor enabled again |
| http://<ip>:8080/stop | |

Check on ESCON Studio the PWM value, the speed and the enable state after each line. For a unipolar .edc use mc, c20 ($\approx 100$ rpm), c10 (0 rpm). If mo/c... do nothing, see "USB access" (Section 1.3).

## 3.4 Session test 1: the basic chain (manual)

Goal: check mode, 0 rpm, start, manual commands, rest, stop, save and plot, without any script.

1. py chipiron_session_v3.py full_session mo parameters_Sep26.edc (bipolar file).
2. Phone ID and network: Enter. Read the printed summary: mapping bipolar, rest command c50.0.
3. Expect: Status: STOPPED; Mode command 'mo' sent; choice 1; Motor to 0 rpm: sending 'c50.0' and the 30 s wait.
4. At the > prompt: c55.0, then c45.0, then x, then d, then e. Watch ESCON Studio after each.
5. Press Enter: motor sent back to rest, acquisition stopped, ls lists the files.
6. Save choice 4, the date and time printed by ls. Check the folder Phonefleet_data/date/time has five files; plot with chipiron_plotting_v3.py (Usb, decoded, both, with the .edc) and check the steps.

Also test py chipiron_session_v3.py save_mode and fetching an old acquisition.

## 3.5 Session test 2: constant speed, ramps and steps

1. py chipiron_session_v3.py full_session mo parameters_Sep26.edc (bipolar; with a unipolar .edc use mc).

2. Choice 3. First question (existing CSV): Enter, to build one.
3. The builder starts (Step A): mode 1 (manual). Build a *short* program, e.g. hold 100 rpm 10 s (menu 1); ramp 100 → 300 rpm in 20 s (menu 2); linear staircase 100–300 rpm, 3 plateaus of 5 s (menu 3); menu 9; rest before 5 s, rest after 5 s. Check the FINAL PROGRAM summary and that there is no WARNING. The CSV is saved as `programs/session_<date>.csv`.
4. The session prints the Step B command, then the 0 rpm command and the 30 s wait, then starts the acquisition and replays the program: one line per command `[ t / total s] c<duty> ~rpm kind segment`.
5. At the end: `Motor sent back to rest (c50.0)`, `Script finished with return code 0`, stop, `ls`, save.
6. Verify on ESCON Studio and on the plot (speed vs. time) that the plateaus and ramps are followed, that the limiter does not flatten them, and that the speed returns to 0.

Repeat with a logarithmic staircase, an up/down cycle (menu 4) and a repeated program (menu 5). For a long ramp check the speed ramp limit of the controller (Section 5).

### 3.6 Session test 3: a ramp-planner CSV

1. `py chipiron_ramp_planner_v3.py -edc parameters_Sep26.edc -unit rpm -type linear -start 100 -end 400 -points 4 -time 10 -folder ramps`.
2. Session as in test 2, choice 3, and answer the first question with the CSV path. No program is built; Step B runs it with `-csv -yes`.
3. Check that the executed duties are the CSV's, and that a CSV planned with another mapping is refused.

### 3.7 Session test 4: oscillation, firmware mode

Needs a bipolar `.edc` and Arduino mode `mo`.

1. `py chipiron_session_v3.py full_session mo parameters_Sep26.edc`; choice 2; oscillation mode `firmware`.
2. Start gentle: mean 0, amplitude 200 rpm, frequency 0.05 Hz (peak acceleration 63 rpm/s, far below 2000), 3 cycles (60 s). Expect `Ramp check OK` and, in the script output, `o50.00,10.00,0.0500`.
3. Watch the motor start from 0, going *up* first (positive speed), then the sine on ESCON Studio.
4. Repeat with 0.1 Hz / 100 rpm, then 0.5 Hz / 300 rpm (peak acceleration $\approx 940$ rpm/s). Try an amplitude that exceeds the limiter (e.g. 600 rpm at 0.5 Hz $\approx 1900$ ok, 800 at 0.5 Hz not) to see the warning and the suggested maximum amplitude.
5. Try a mean $\neq 0$ (e.g. 200 rpm, amplitude 100 rpm) and check the sine is centred there.
6. Small amplitudes: 30 rpm ($\pm 1.5$ %) is expected not to reverse the speed on the lab motor (dead zone); check on Chipiron.
7. Plot the speed: the amplitude, the frequency and the offset must match.

NB: oscillations in stream mode are les smooth and therefore not recommended. Stream is only useful with a unipolar `.edc` or in `mc`, where firmware mode is not possible.

### 3.8 Fault and safety tests

- Session with `mc` and the bipolar `.edc`: must stop with "rests at 10 % duty but the .edc gives 0 rpm at 50 %".

- Firmware-mode oscillation with `mc`, or with amplitude beyond $\pm 800$ rpm: ERROR before anything starts.
- Choice 3, Step A with an out-of-range block: ERROR, block not added.
- Unplug the Arduino USB after the mode is sent: commands must fail visibly; the final rest command prints a "STOP THE MOTOR MANUALLY" message if it cannot be sent.
- Ctrl-C during a run: the script prints "Interrupted", sends the rest command, the session stops the acquisition.
- After a reset of the Arduino (re-plug the USB) the motor must be *disabled* until `mo`, `mc` or `e`.

# 4 Sine skewing debugging (lab motor, 21–22 October 2026)


**Summary**

A commanded sine $0 \pm 600$ rpm at 0.1 Hz came out with an amplitude of only about 200 rpm, *also on the live ESCON Studio display*, so it is not a plotting or Python artefact. The cause is the ESCON **speed controller P gain = 1** (the ESCON default is 200). With such a weak gain the closed speed loop behaves as a first-order low-pass filter with time constant $\tau \simeq 4.6$ s, whose corner frequency (0.034 Hz) is far below 0.1 Hz.
On the lab motor $\tau \times$ gain $\simeq 4.6$ s for gains 1 to 20. At gain 20 ($\tau \simeq 0.25$ s) the 0.1 Hz sine is faithful ($\times 0.99$) and the 0.5 Hz sine comes out at $\times 0.79$ with a $38°$ lag, which matches what the step response predicts. The ESCON "averaged" analog outputs add no measurable lag and divide the measurement noise by about 4: use them for the data (Section 4.4). Everything below was measured on the *lab* motor and load; the gain needed on Chipiron must be re-measured (Section 4.5).


## 4.1 The model: a first-order lag

The closed speed loop (ESCON speed controller + motor + load) is modelled by

$$\tau\,\dot{y} + y = K\,u, \qquad \text{transfer function } H(s) = \frac{K}{\tau s + 1},$$

with $u$ the commanded speed, $y$ the actual speed, $K$ the DC gain (measured $\approx 0.99$) and $\tau$ the time constant. Two standard consequences are used throughout:

| Test signal | Response |
|---|---|
| Step at $T_0$ | $y(t) = y_0 + (y_f - y_0)(1 - e^{-(t-T_0)/\tau})$ for $t \geq T_0$. 63 % reached after $\tau$, 90 % after $2.3\,\tau$, 10–90 % rise time $2.197\,\tau$ |
| Sine of frequency $f$ | steady state: amplitude ratio $G = 1/\sqrt{1 + (2\pi f\tau)^2}$ and phase lag $\varphi = \arctan(2\pi f\tau)$. Corner frequency $f_c = 1/(2\pi\tau)$ (here $G = 0.707$, lag $45°$) |

**Largest acceptable $\tau$** Asking for an amplitude ratio $G \geq G_0$ at frequency $f$ gives $(2\pi f\tau)^2 \leq 1/G_0^2 - 1$, i.e.

$$\tau \leq \tau_{\max}(f) = \frac{\sqrt{1/G_0^2 - 1}}{2\pi f}.$$

For $G_0 = 0.9$: $\sqrt{1/0.81 - 1} = 0.484$ and $0.484/(2\pi) = 0.0771$, so $\tau_{\max} = 0.077/f$ (for $G_0 = 0.95$ it is $0.052/f$). At $f = 0.1$ Hz this is $\tau_{\max} = 0.77$ s, the value printed by the step-fit cell. Since $\tau \simeq K_\tau/\text{gain}$ with $K_\tau \simeq 4.6$ s on the lab motor, the gain needed is about $K_\tau/\tau_{\max}$:

| $f$ (Hz) | $\tau_{\max}$ for 90 % (s) | gain needed (lab motor) |
|---|---|---|
| 0.1 | 0.77 | $\approx 6$ |
| 0.3 | 0.26 | $\approx 18$ |
| 0.5 | 0.15 | $\approx 30$ |
| 1 | 0.077 | $\approx 60$ |

Only gains 1, 5 and 20 were actually resolved (Section 4.3); the rows above 20 are an extrapolation and must be confirmed with sines.

**Other limits on a sine.** Even with a fast loop, the peak acceleration of $A\sin(2\pi f t)$ is $A\,2\pi f$. For $A = 600$ rpm: 377 rpm/s at 0.1 Hz, 1885 rpm/s at 0.5 Hz (just below the 2000 rpm/s ramp used, no flattening was seen) and 3770 rpm/s at 1 Hz, which exceeds it: raise the ramp (`chipiron_patch_edc_v3.py -ramp`) or reduce the amplitude to at most $2000/(2\pi) \approx 318$ rpm at 1 Hz. The torque needed is $J\,\alpha$ ($J$ = total inertia), so it also grows with the load.

## 4.2 The two fit cells in `chipiron_plotting_v3.py`

two optional debgging cells on the decoded A2 speed in rpm (the `.edc` is needed for the volts → rpm conversion). ask a fit window and a free *gain label* for each selected acquisition, print a summary, save a one-row CSV next to the raw data (`<stamp>_stepfit.csv` or `<stamp>_sinefit.csv`, never overwritten) and a figure (data, fit, residuals).

**Step fit (hold / step test).** Fits $y_0$, $y_f$, $T_0$, $\tau$ of the step law above by least squares. It prints $T_0$, $\tau \pm \sigma$, the 10–90 % rise time, the corner frequency, the speed reached versus the commanded one, and the amplitude ratio and lag a sine of a chosen frequency would have (plus the largest $\tau$ keeping 90 %). Use a window that *starts at rest* and ends *before* the return to rest. A summary table and a $\tau$-versus-gain figure are made when several acquisitions are selected.

**Sine fit (sine test).** The sine starts at an unknown time $T_0$ from rest (level $c$). The model, with $t' = t - T_0$ and $\omega = 2\pi f$, is

$$y = c + K\Big[\text{mean}\,(1 - e^{-t'/\tau}) + A\,\frac{\sin\omega t' - \omega\tau\cos\omega t' + \omega\tau\, e^{-t'/\tau}}{1 + \omega^2\tau^2}\Big], \quad t \geq T_0,$$

i.e. the exact solution of the first-order law for a commanded mean $+ A\sin\omega t'$, including the start-up transient. Free parameters: $\tau$, $T_0$, $c$ and $K$ ($K$ can be fixed to 1). A second, model-free fit $y = c + a\sin\omega t + b\cos\omega t$ on the steady state ($t \geq T_0 + \max(1/f, 3\tau)$) gives the measured amplitude ratio $\sqrt{a^2 + b^2}/A$ and the lag relative to the command (using the fitted $T_0$). If the two results disagree by more than 15 % in amplitude or $15^\circ$ in lag, the loop is not first order (or $T_0$ is poorly determined) and a warning is printed. Also checked: peak acceleration versus the ESCON ramp, at least 4 samples per cycle, the window starting at rest.

**Limits of both fits**

Telemetry is 4 Hz ($dt = 0.25$ s): a step with $\tau \lesssim 0.5$ s has fewer than 2 samples on its rise and $\tau$ is *not resolved* (the fit returns a value of the order of one sample period, or a degenerate one with $\sigma =$ nan). Quantisation noise (steps of 78 rpm, Section 4.4) makes the printed $\sigma$ a lower bound. Use sines (fit of amplitude ratio and lag) or the ESCON data recorder (about 20 ms sampling) for fast loops. A 600 rpm step also takes at least 0.3 s if the 2000 rpm/s ramp is active, whatever the gain.

## 4.3 Log of the tests and what each one gave

All tests: lab motor, bipolar `.edc` with AO shift (A1 = current, A2 = speed), speed ramp 2000 rpm/s, current limit 10 A, $V_{\mathrm{ref}} = 4.88$ V. Times are the file stamps. "Raw" = ESCON analog output "Actual Speed" (code 4), "averaged" = "Actual Speed Averaged" (code 5).

| When | Test | Result | Conclusion |
|---|---|---|---|
| 20/10 | motor held at 600 rpm in ESCON Studio, compared with the Arduino readings | A1 ← AO1 = current, A2 ← AO2 = speed; $V_{\mathrm{ref}} = 4.88$ V | wiring and $V_{\mathrm{ref}}$ set in the scripts (Section 3.2) |
| 21/10 16:29:14 | sine $0 \pm 600$ rpm, 0.1 Hz, gain 1 (repeated: same on the live ESCON display) | fitted amplitude $\approx 194$ rpm (peaks about $\pm 240$ rpm) | not an artefact of Python or plotting |
| 21/10 16:33:31 | hold 600 rpm for 120 s after 30 s at rest, gain 1 | $\tau = 4.62 \pm 0.23$ s, reaches 596 rpm (99.3 %), rms residual 43 rpm | predicts $\times 0.32$ at 0.1 Hz $= 190$ rpm: matches the sine |
| 21/10 16:41:05 | 30 s rest, then ramps 0→600 rpm in 0.3, 1, 5, 50, 100 s, 10 s rest between, gain 1 | short ramps barely move the motor (peaks about 75, 157, 235 rpm); the 50 and 100 s ramps are followed with a few seconds of lag and reach 550–600 rpm | consistent with a low-pass, rise is exponential, not a ramp limit |
| 22/10 09:51:08 | hold 600 rpm, gain 5, raw | $\tau = 0.84$ s | $\tau \times$ gain $= 4.2$ |
| 22/10 09:57:26 | same, gain 20, raw | $\tau = 0.25 \pm 0.05$ s, 595 rpm | $\tau \times$ gain $= 4.9$; only about one sample on the rise: at the resolution limit |
| 22/10 10:00:26 | same, gain 50, raw | fit gives 0.011 s, $\sigma =$ nan | **unresolved** (step finishes within a sample) |
| 22/10 10:04:06 | same, gain 100, raw | $0.11 \pm 0.03$ s | **unresolved** |
| 22/10 11:20:30 | sine $0 \pm 600$ rpm, **0.5 Hz**, gain 20, raw | $\times 0.79$, lag $37°$, $\tau = 0.24$ s, residual about $\pm 100$ rpm | matches the prediction from the step ($\times 0.79$, $38°$) |
| 22/10 11:24:23 | same, averaged | $\times 0.79$, lag $38°$, $\tau = 0.25$ s, residual about $\pm 25$ rpm | same dynamics, noise divided by 4 |

| When | Test | Result | Conclusion |
|---|---|---|---|
| 22/10 11:35:22 | sine **0.1 Hz**, gain 20, raw | $\times 0.99$, lag $16°$, $\tau = 0.46$ s | fixed for slow sines |
| 22/10 11:29:23 | same, averaged | $\times 0.99$, lag $13°$, $\tau = 0.36$ s | prediction was $9°$; at 0.1 Hz the lag hardly constrains $\tau$ |
| 22/10 11:39:11 | hold 600 rpm labelled gain 5, raw | $\tau = 0.26$ s | looks like gain 20 ($\approx 0.23$ s), not gain 5 ($\approx 0.9$ s): **check that the gain was written to the controller** |
| 22/10 11:43:24 | same, averaged | $\tau = 0.25$ s, within a few rpm of 596 rpm | same $\tau$ as raw: averaging adds no lag at this resolution |

**Overall conclusions.**

1. The skew of the sines is the speed loop being a low-pass filter ($\tau = 4.6$ s at gain 1). It is neither the PWM mapping, nor the current limit (current stays below about 1 A), nor the ramp (the step response is exponential, and the static gain is correct: 99.3–99.7 % of 600 rpm at every gain).
2. $\tau \propto 1/\text{gain}$ was verified from gain 1 to 20 ($\tau \times \text{gain} = 4.2$–$4.9$ s). The step-derived $\tau$ predicts the 0.5 Hz sine amplitude and phase to within the fit uncertainty, which validates the first-order model in this range.
3. At gain 20 the sine is good up to about 0.1 Hz ($\times 0.99$) and loses 21 % of its amplitude at 0.5 Hz. For 0.5–1 Hz a gain of order 30–60 is expected to be needed, *not yet verified*.
4. Step tests cannot resolve $\tau$ below about 0.5 s with 4 Hz telemetry, so gains of 50 and 100 were not characterised.
5. The "averaged" outputs are preferable for data (Section 4.4).

## 4.4 Speed quantisation and the ESCON "averaged" outputs

**What is observed on the raw speed.** The raw A2 speed takes values in steps of about 78 rpm (levels 0, 78, 157, 236, 314…). During a 600 rpm hold it alternates between about 552 and 630 rpm (mean 594–598 rpm), with rare spikes near 470 and 710 rpm; the rms scatter around the fit is 36–44 rpm. This is not the Arduino ADC (one ADC count is only 2.4 rpm, so 78 rpm is 33 counts).

**Probable origin (hypothesis, not verified).** ESCON computes the speed from encoder edges counted in each control cycle. With the `parameters_Sep26.edc` encoder (1024 lines, hence 4096 edges per turn), one edge per cycle of 0.187 ms corresponds to $60/(4096 \times 0.187 \times 10^{-3}) = 78.3$ rpm, and 0.187 ms is the time base of the ESCON data recorder in the same file. The raw output is then the speed of *one single control cycle*: at a true speed between two levels it jumps from one cycle to the next between the two neighbouring multiples of 78 rpm, and the true value is the time average of these jumps. The current output is noisy for the same kind of reason (PWM ripple). To be checked against the maxon documentation and the encoder of the motor really used.

**What the "averaged" output does.** `Actual Speed Averaged` (analog output function 5; `Actual Current Averaged` is 3, the raw ones are 4 and 2) is a filtered version of the same quantity, computed *inside the controller from all its control cycles*. The exact filter (window type and length) is not documented in the file and is **not known/ to be checked**: to be read in the maxon manual or asked to maxon. What the tests show is that its extra delay is too small to see (Section 4.3).

**Doubts about averaging afterwards by hand**

1. **The Arduino sees one sample every 250 ms.** Between two samples the ESCON runs about $250/0.187 \approx 1300$ control cycles. Reading the raw output means picking *one* cycle out of 1300 (a random draw between the two levels, i.e. aliasing); reading the averaged output means getting a mean over many cycles at every sample.
2. **Averaging $N$ raw samples costs time resolution.** Noise falls as $1/\sqrt{N}$: to go from $\pm 39$ rpm to $\pm 10$ rpm you need $N \approx (39/10)^2 \approx 15$ samples, i.e. a window of about 4 s. This is much longer than $\tau = 0.25$ s or the 2 s period of a 0.5 Hz sine, so the dynamics we want to see would be erased. A shorter window (say 4 samples, 1 s) still smears the response by an amount comparable to $\tau$.
3. **It cannot be done afterwards.** Whatever happened between two 4 Hz samples is lost for good: a fast average can only be made inside the controller (or with a faster acquisition).
4. **It does not bias the mean.** Raw and averaged outputs give the same mean speed (594–598 rpm versus 596 rpm).

**Measured effect (gain 20 sines and gain-5-labelled holds).**

| | **raw** | **averaged** |
|---|---|---|
| sine 0.5 Hz | $\times 0.79$, lag $37°$, residual about $\pm 100$ rpm | $\times 0.79$, lag $38°$, residual about $\pm 25$ rpm |
| sine 0.1 Hz | $\times 0.99$, lag $16°$ | $\times 0.99$, lag $13°$ |
| hold 600 rpm | alternates 552 / 630 rpm, $\tau = 0.26$ s | within a few rpm of 596 rpm, $\tau = 0.25$ s |

At 0.5 Hz one degree of phase is 5.6 ms; the $1°$ difference, with an uncertainty of a few degrees, bounds the extra delay of the averaging to about 10–20 ms. This is a statement at 0.5 Hz and $\tau \approx 0.25$ s only.

**CCL** Use the averaged outputs for the data (set AO1 to `Actual Current Averaged` and AO2 to `Actual Speed Averaged` in ESCON Studio. Ideally, run tests twice and save both raw/mean outputs, or ifnd a way to export both thorugh ESCON? Keep the raw outputs for diagnosing the loop itself, and repeat one raw-versus-averaged comparison (same sine, same gain) whenever the frequency range goes well above 0.5 Hz.

## 4.5 Procedure for the real motor and load

The time constant is roughly (total inertia) / (gain): a heavier load needs more gain than the lab motor. The aim is the *lowest* gain that gives the required amplitude fidelity at the highest frequency, with a safety margin.

**Before raising the gain**

Rotor and load free to turn, a hand on the supply switch, current limit set (4–5 A suggested for Chipiron), small speeds first. A gain that is too high gives overshoot, ringing or buzzing;

stop with Ctrl-C (the run script sends the rest command) and go back to the previous value. Battery plugged last.

1. **Fix the targets.** Highest frequency $f_{\max}$ and amplitude $A$ of the experiments. Compute $\tau_{\max} = 0.077/f_{\max}$ (for 90 % amplitude) and the peak acceleration $A\,2\pi f_{\max}$; check it against the ESCON ramp (raise it if needed) and against the torque and current available.
2. **Set the measurement chain first.** Re-check the A1/A2 wiring and measure $V_{\mathrm{ref}}$ (Section 3.2); use the `.edc` with the AO shift; choose the averaged outputs after one raw-versus-averaged check; note the `.edc` file name.
3. **Calibrate the load.** At gain 1 and then 5: a hold step small enough not to be limited by the ramp (say 100–200 rpm, 30 s at rest first, 30 s or more at speed), fit $\tau$ with the step cell, compute $K_\tau = \tau \times$ gain and the gain needed, $K_\tau/\tau_{\max}$.
4. **Raise the gain in factors of about 2–3.** While $\tau \gtrsim 0.5$ s use steps; once $\tau$ is below that, switch to sines of amplitude $A$ at $f_{\max}$ and at two or three lower frequencies (at least 10 cycles each, starting at rest) and read the amplitude ratio and lag from the sine fit. For fast steps the ESCON data recorder can be used. Check at each step that $\tau \times$ gain stays about constant.
5. **Stop** at the first gain that meets the target, then add a margin of $\times 1.5$–2. Stop earlier if the step shows overshoot or ringing, if the motor buzzes, if the current peaks approach the limit, or if the torque signal becomes noisy. Only if there is overshoot, adjust the integral time (11.2 ms in the lab file) afterwards; change one parameter at a time.
6. **Validate with the real protocol.** Run the planned sines (and ramps) and compare the measured amplitude and phase of the speed channel with the commanded values; for the rheology, use the measured speed, not the commanded one, in the analysis.
7. **Write everything down.** Gain, Ti, ramp, current limit, `.edc` name, raw or averaged outputs, load and sample, and the fit CSVs, in the metadata of the experiment. Repeat the check when the load or the sample changes. ESCON auto-tuning can give a starting value but moves the motor: only with a free rotor.

## 4.6 Open points

- Can we export mean/raw files at the same time Through ESCON?.
- Is the 2000 rpm/s ESCON ramp active? The 0.5 Hz sine (1885 rpm/s) was not flattened, but steps at gain 50–100 are compatible with a ramp limit. A step of 100–200 rpm at high gain would tell.
- Exact filter of the "averaged" outputs; origin of the 78 rpm quantisation (hypothesis above).
- Behaviour above gain 20 (stability, overshoot) and with the real load; the first-order model must be re-checked there (residuals of the fits).
- The lag of the 0.1 Hz sine (13–16° measured, 9° predicted) is within what the fit can resolve, but could hide a small extra delay.

# 5 Current limitations of the scripts

**Testing status**

**The LoRa part has not been tested yet** (`-link lora`, `LoraLink`, `USE_LORA 1` in the firmware, the bridge Arduino). The v3 scripts were only checked offline against a simulated phone (program building, CSV, oscillation commands, patching, decoding); the session, `mobile_v3.py`, `tsync_v3.py` and the plotting script were not run end to end, and nothing was run on Chipiron or on the motor.

- **No acknowledgement.** The Arduino never answers: HTTP 200 only means the phone got the command. A lost or garbled command is not detected (the session prints the phone's reply).
- **No log of the commands sent.** Run/oscillate print their commands but nothing is stored with the phone files, and the phone clock and the laptop clock are not aligned automatically (`tsync_v3.py` is optional and not integrated). Keep `programs/*.csv` with the data and align on the start of the motion.
- **Timing.** Commands are sent from Python over Wi-Fi: jitter of tens of ms is expected. Ramps are staircases (`-dt` $\geq 1$ s recommended); **stream mode** oscillation is coarse and only for periods $\gtrsim 20$ s. The duty resolution is 0.39 % (about 7.8 rpm with 20 rpm per %) and a ramp can have several equal steps.
- **Oscillation.** Firmware mode needs `mo` and a bipolar `.edc`; the sine starts at the mean going up; the firmware silently clamps amplitude and mean to 10–90 % (the scripts check before). No phase control. A low-amplitude dead zone exists on the lab motor ($\pm 4$ % needed to reverse the sign).
- **ESCON ramp limiter.** The checks assume a start at rest and a simple model of the limiter; with a very slow ramp on Chipiron (e.g. 10 rpm/s) plateaus need long times. The `.edc` given to the scripts must be the one loaded in the controller: nothing verifies it.
- **Offset adjust.** An offset stored in the `.edc` is ignored by the mapping unless enabled in the file; the effect (additive) is an assumption to check in ESCON Studio.
- **Rest at the beginning.** Only the session sends the rest command (0 rpm, then 30 s) before the start. Run and oscillate used standalone do not; the 30 s wait and the 30 s default rest before a program add up in the recording.
- **A1/A2 and $V_{\mathrm{ref}}$.** The wiring is not detected: the scripts assume A1 = current (AO1), A2 = speed (AO2), as found in the lab; check it on Chipiron. The ADC reference `ADC_VREF` must be **measured** (4.88 V on the lab Arduino, not 5 V) and re-measured if the power source changes; 10 bits are assumed. Lost bytes give `low8` cycles whose ADC value is only known modulo 256.
- **No dry run.** There is no option to "run" a program without sending (use `-build-only`); a run with the phone unreachable fails at the first command.
- **Concurrency.** Two commands arriving within 5 ms at the Arduino can be merged; the command buffer is 32 characters. `usb <command>` in `mobile_v3.py` is sent to all active phones.
- **Phone/software.** `-song` needs the `play` command and `test.mp3`; file transfers time out after 60 s; the HTTP routes depend on the Gobannos version; `tsync_v3.py` needs UDP ports 5000/5001 open on the laptop firewall.

# 6 Field warnings and notes (with the real Chipiron)

## 6.1 Messy notes

**A1 vs A2:** on the real Chipiron (according to Stéphane) the speed byte pair arrives first, then 50 ms later the current pair, then 150 ms later the next loop; to be checked on the field (Section 3.2).

Need to check on the field that:

1. the imposed current does not depend on the exact value of the battery voltage;
2. the motor range is properly configured for the tests we want to do;
3. we can decode and re-code the USB data properly.
4. $V_{\mathrm{ref}}$ of the Arduino (multimeter between GND and 5V, powered as in the experiment) and the A1/A2 wiring;
5. the **range where the sine is not skewed**: find the amplitude/frequency domain where the measured speed stays a clean sine (check the ramp possibility, i.e. the ESCON speed ramp against the peak acceleration $A\,2\pi f$, and the current limitations).

Max current for the motor: 9 A. Caution: we will blow up mechanics and other stuff way before 9 A, so maybe set a 4–5 A limit somewhere.

Plug the battery at the END.

Chipiron: PWM ratio 2460 rpm $\leftrightarrow$ 90 % (previous Stéphane's config, 2025/26), speed ratio $\frac{2460}{4}$ rpm/V. Current ratio: $\frac{30}{4}$ A/V. (Different from the lab `.edc`: always use the `.edc` of the controller.)

If the ramp acceleration is fixed to a low 10 rpm/s value on Chipiron, make sure to spend enough time/measuring point.

**Low-amplitude limit.** For oscillation tests, with an amplitude of 1.5 % duty ($\pm 30$ rpm) the speed never changed sign. The sign reverses from about $\pm 4$ % duty (about 80 rpm). Small amplitudes are therefore not reachable with the current setup on the lab motor and there is a dead low-amplitude zone where it does not move. To be checked on Chipiron!

## 6.2 Oscillation tests: physical target

**Our physical target for an oscillatory rheology test is a sinusoidal deformation (strain), not a sinusoidal rotational speed as such.**

If the vane angle $\theta$ is proportional to the deformation $\gamma$, then a target

$$\gamma(t) = \gamma_0 \sin(2\pi f t)$$

requires a sinusoidal angle. Angular velocity is then at the same frequency but shifted by $90°$:

$$\dot{\theta}(t) = 2\pi f\, \theta_0 \cos(2\pi f t).$$

The relationship between vane angle and sample strain must be calibrated for the actual Chipiron geometry.

If $\gamma = K\theta$, the required angular-speed amplitude is

$$\dot{\theta}_0 = 2\pi f \, \frac{\gamma_0}{K}.$$

In rpm, this is

$$\mathrm{rpm}_0 = 60 f \, \frac{\gamma_0}{K}.$$

The scripts command a sinusoidal *speed* mean + amp $\sin(2\pi f t)$; for a strain-controlled test the speed has to be programmed as the cosine above, which is not yet possible: firmware mode starts at the mean going up (a sine), and there is no phase option.